ARQUITECTURA DE SOFTWARE · PARTE I · SUSTENTACIÓN

La Gran Liga de Apuestas de Medianos EIA

♠ ♥ ♦ ♣

Salchipapa Maker Racing S.A.S.

Estructura, lógica y decisiones: diseño de la plataforma de apuestas mutuas (Pari-Mutuel) sobre carreras de medianos

100K	10K/S	< 1 S	3 ADR
usuarios en pico	apuestas en ráfaga	respuesta al apostar	decisiones registradas

Octubre 2026 · Documentación de arquitectura — sin implementación de plataforma

02 · Contexto del negocio y el desafío de Dyllan

De una hoja de cálculo a un sistema profesional que no pierda dinero ni credibilidad

- El negocio nace como torneo universitario informal entre carreras («medianos»), con apuestas entre compañeros usando **EIA Coins**, la moneda virtual del ecosistema.
- El problema actual es operativo y contable: carreras, apuestas y saldos manejados de forma manual, sin cierre automático, sin trazabilidad y sin defensa contra duplicados.
- El enunciado exige un sistema profesional con **nueve capacidades**: usuarios, medianos, carreras, apuestas, billetera, resultados y premios, notificaciones, estadísticas y auditoría.
- La directriz económica de la dirección es parte del problema:

> *«No quiero pagar una infraestructura de Netflix para una carrera de cinco medianos.»*

Capacidades principales (PDF pág. 3–8)

Usuarios y auth | Medianos | Carreras | Apuestas | Billetera EIA Coins | Resultados y premios | Notificaciones | Estadísticas | Auditoría y fraude

Riesgos identificados desde el inicio

- **Latencia vs. concurrencia:** < 1 s con 10.000 apuestas/s.
- **Incertidumbre de red:** un timeout no significa fallo (exige idempotencia).
- **Sobreingeniería:** complejidad que el negocio no paga.

03 · De las cifras del enunciado a la estructura

Cada número obligó una decisión concreta de estructura (PDF pág. 8–9)

Cifra clave	Decisión de estructura que obligó
`10.000 apuestas/s` sostenidas 30 s (300.000 en la ráfaga)	Escrituras particionadas por usuario y bloqueo a nivel de **fila**, nunca una cerradura global que serialize al sistema entero.
Respuesta al apostar **`< 1 s`**	Cero RPC síncronos de más en la ruta crítica: lo que puede tardar (cuotas, notificaciones, reportes) sale de ella por outbox y eventos.
`Cero` tolerancia a pérdida de datos	Saldo, apuesta y evento se escriben en **una sola transacción**: no existe el estado «cobré sin apostar».
Liquidación **`< 15 min`** para > 1.000.000 de apuestas	Cálculo global único + ~1.000–2.000 lotes con 10 workers en paralelo; jamás dentro de una petición HTTP.
`100.000` usuarios en el pico	Gateway y núcleo **stateless** y replicables; lecturas masivas en réplicas, no en el primario.
Estadísticas con tolerancia de minutos	Analítica segregada en su propio proceso y almacén OLAP: consistencia eventual, deliberadamente.
Auditoría **`≥ 5 años`**, solo lectura	Almacén append-only con inmutabilidad física, separado del OLTP operativo.

04 · Lógica de negocio traducida en diseño

Pari-Mutuel: la casa retiene 10 % y reparte el 90 % — esa regla manda cuatro decisiones

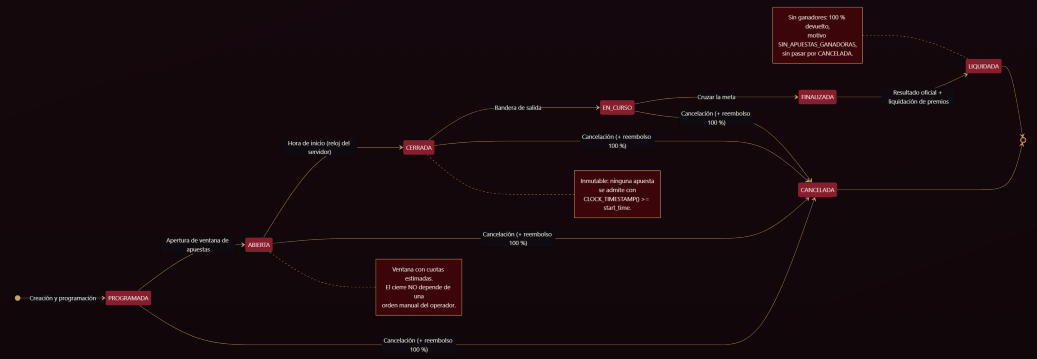
Por qué esta lógica exige esta estructura

- La casa solo reparte: el dinero debe conservarse **exacto al centavo** → aritmética en centavos enteros, nunca coma flotante (slide 14).
- La cuota se sella en el instante del cierre → el cierre no es un botón del operador: es el **reloj del servidor** validado dentro de la transacción (slide 10).
- El mismo usuario puede apostar varias veces en una carrera legítimamente → la deduplicación solo actúa sobre **reintentos**, por clave idempotente (slide 13).
- Siete estados con transiciones guardadas → **ninguna transición es manual**; la FSM de la derecha es la única verdad del ciclo.

Reglas que no se negocian

- Las cuotas en vivo mientras `ABIERTA` son **estimación informativa**, no garantía: la cuota definitiva se calcula al cierre.
- Si nadie acierta al ganador: devolución del 100 % — decisión formal, slide 15.

Ciclo de vida de la carrera (FSM)



05 · Invariantes: la lógica que jamás se viola

Cada mecanismo del sistema existe para sostener exactamente uno de estos cinco

Invariante	Mecanismo que lo garantiza	Dónde se justifica
1 · Nunca saldo negativo	Bloqueo de fila `FOR UPDATE` + restricción de la propia base que rechaza saldos negativos: la integridad la garantiza la BD, no el código de la app.	ADR-003 · doc 05
2 · Nunca apuesta tras el cierre	`INSERT` condicionado con `CLOCK_TIMESTAMP()`: si el reloj ya pasó, 0 filas → ROLLBACK → HTTP 409, irrevocable.	doc 03 · slide 10
3 · Conservación exacta de la bolsa	Mayores restos en centavos enteros con redondeo bancario: comisión + premios = bolsa, exactamente al centavo.	ADR-003 · slide 14
4 · Ni doble gasto ni premio doble	Claves únicas en tres niveles: apuesta, premio y evento procesado — el reintento choca contra el índice y devuelve lo ya hecho.	ADR-003 · slide 13
5 · Auditoría inmutable	Outbox en la misma transacción + registro de solo agregar retenido ≥ 5 años: nunca se borra ni se actualiza.	doc 04 · doc 07

Regla de lectura para el jurado

Si preguntan «¿por qué existe `FOR UPDATE`?» o «¿por qué el outbox?», la respuesta está en esta tabla: sostiene un invariante. No acumulamos tecnologías; cada mecanismo responde a una violación que queremos hacer **imposible**.

06 · Decisión 1: Estilo Arquitectónico (ADR-001)

Cuatro estilos evaluados; se adopta la Arquitectura Híbrida (Núcleo Modular + workers desacoplados)

Estilo	Consistencia en la ruta crítica	Aislamiento de fallos / carga	Costo y operación	Veredicto
Monolito tradicional	ACID local, pero sin fronteras formales	**Pobre:** un fallo en notificaciones o reportes degrada el hilo transaccional	Bajo costo inicial, deuda de diseño creciente	**Descartado**
Monolito modular completo	ACID local en todas las capacidades	Medio: todo escala junto, sin segregación de analítica masiva	Simple; pero arrastra OLAP pesado al núcleo	Parcial
Microservicios puros (9 servicios)	**Saga/2PC** entre apuesta y billetera en < 1 s	Excelente aislamiento, alta complejidad emergente	**Alto:** 9 despliegues, 9 BDs, tracing, mTLS, discovery	**Descartado**
Arquitectura híbrida	**ACID local** para débito + apuesta (una transacción)	Notificaciones, analítica y liquidación **fuera** de la ruta crítica	1 núcleo replicable + 1 Kafka + workers ligeros	**ADOPTADO**

Ganancia 1

Sin 2PC ni Sagas en el flujo de admisión.

Ganancia 2

La caída de notificaciones no frena apuestas.

Límite asumido

Disciplina modular obligatoria y contención de BD gestionada por particiones y fila-locks.

07 · DESCARTE JUSTIFICADO DE ALTERNATIVAS

Por qué no microservicios puros ni monolito clásico — con argumentos técnicos, no de moda

✗ Microservicios puros

- Separar físicamente **apuestas** y **billetera** destruye la transacción local: exige **Sagas con compensación** en la ruta de 1 segundo.
- Más puntos de fallo y latencia de red por apuesta (10.000 veces por segundo).
- Complejidad operativa desproporcionada: service discovery, mTLS, tracing, esquemas de despliegue × 9.
- Contra la directriz de Dyllan: costo y equipo de operación que el negocio no sostiene.

✗ Monolito tradicional

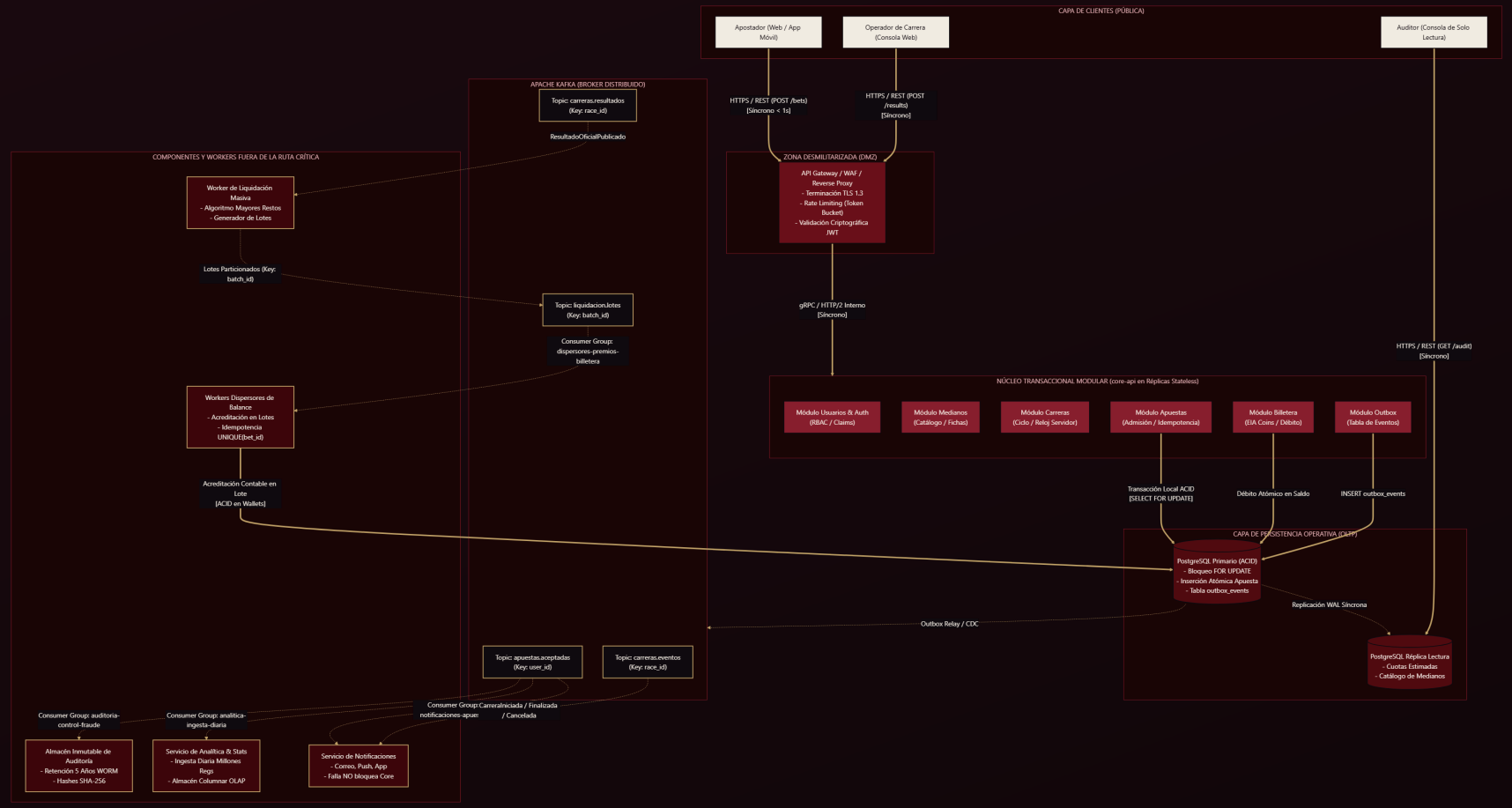
- Sin fronteras: un proveedor de notificaciones lento **agota hilos/conexiones** y bloquea apuestas legítimas.
- La ingesta analítica de millones de registros/día competiría por los mismos bloqueos y IOPS que la ráfaga del cierre.
- Degrada el invariante de disponibilidad sin aportar ninguna ventaja de consistencia adicional.

✗ ¿Y el monolito modular completo?

Es el paso intermedio lógico; se adopta **parcialmente**: el núcleo es modular, pero analítica y liquidación se segregan porque su carga y sus fallos son de otra naturaleza.

08 · Diagrama general de arquitectura

Clientes → DMZ (CDN + Gateway/WAF) → Núcleo Modular transaccional → Kafka y almacenes privados



Zona pública (DMZ)

CDN de estáticos y API Gateway con rate limiting, WAF y terminación TLS 1.3. Ningún servicio ni BD expone puertos a Internet.

Núcleo transaccional

`core-api` stateless y replicable con módulos Usuarios, Medianos, Carreras, Apuestas y Billetera sobre OLTP (PostgreSQL primario + réplica).

Plano asíncrono

Transactional Outbox → Kafka → workers: notificaciones, analítica OLAP, auditoría inmutable y liquidación masiva.

09 · Diferenciación: ruta síncrona (< 1 s) vs. plano asíncrono

Solo cinco comunicaciones son síncronas; todo lo demás fluye por eventos con Outbox

Origen → Destino	Protocolo	Por qué síncrono	Estrategia ante fallo
Cliente → Gateway	HTTPS/REST	El usuario necesita ver si apostó (< 1 s)	Timeout cliente 800 ms + reintento con la misma `Idempotency-Key`
Gateway → `core-api`	HTTP/2 · gRPC	Ruta crítica, latencia interna mínima	Circuit breaker 600 ms → HTTP 504 con estado incierto
`core-api` → OLTP	Nativo BD (pool)	No doble gasto, cierre y saldo atómicos	Timeout 500 ms → rollback y HTTP 503
Operador → `core-api`	HTTPS/REST	Sellado del resultado oficial con confirmación	Reintento idempotente por `race_id`
Auditor → réplica	HTTPS/REST	Consulta interactiva forense	Lectura en réplica; timeout 5 s

Asíncrono vía Kafka (eventos canónicos)

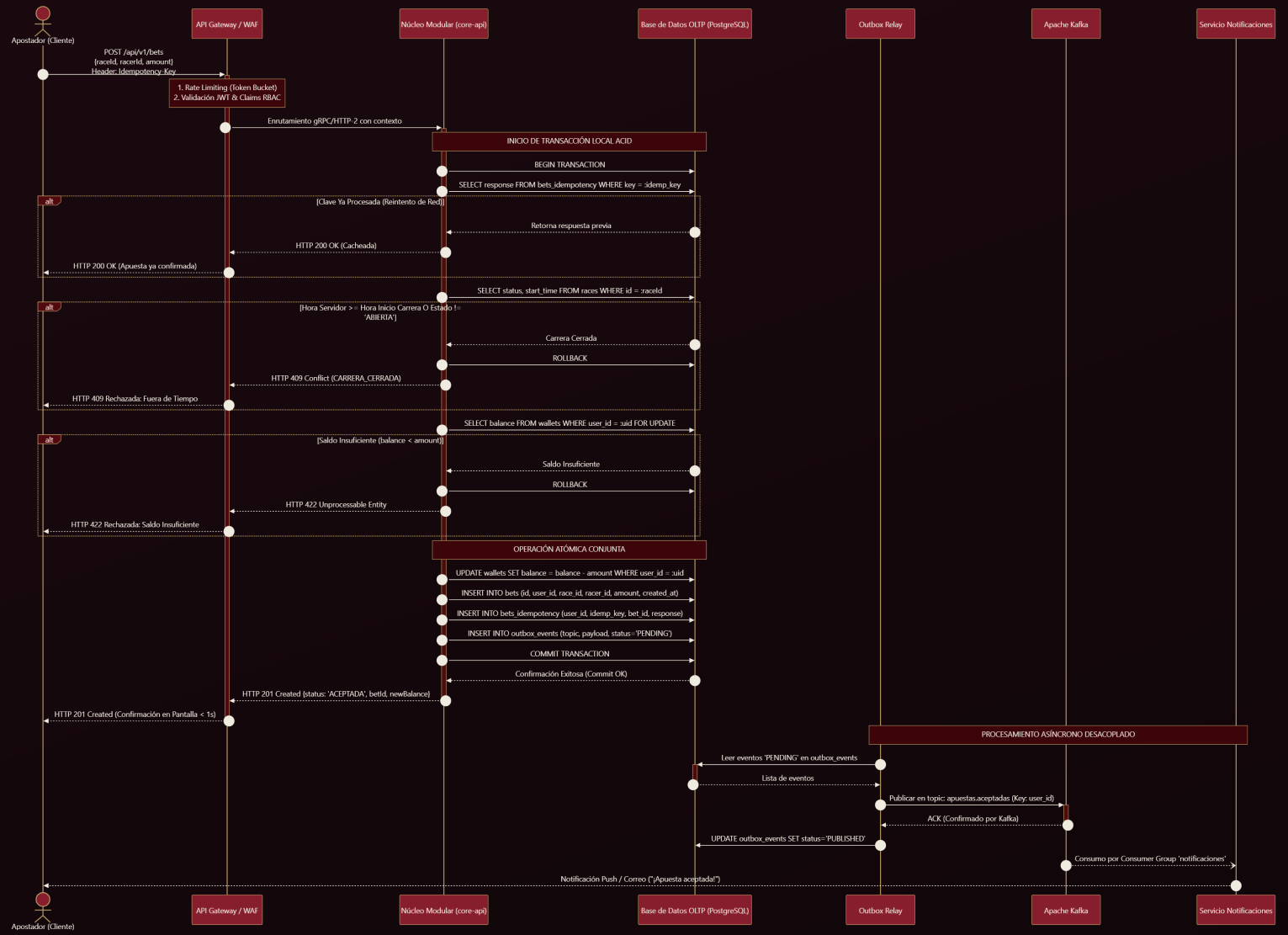
- ApuestaAceptada
- CarreraIniciada
- CarreraFinalizada
- ResultadoOficialPublicado
- LoteLiquidacionGenerado
- PremioAcreditado
- CarreraCancelada
- DevolucionAcreditada

Regla de oro

Kafka **jamás** está en el ciclo de reloj de la petición HTTP: los eventos se publican con **Transactional Outbox** tras el COMMIT. La escritura en el broker no puede costarle milisegundos al apostador.

10 · Flujo secuencial de una apuesta

Rate limiting, idempotencia, bloqueo de fila, transacción ACID local y outbox en un solo ciclo



1 · Borde

Gateway: rate limiting (Token Bucket), validación JWT y claims RBAC antes de tocar el núcleo.

2 · Transacción local

Idempotencia → carrera `ABIERTA` y `CLOCK_TIMESTAMP() < start_time` → `FOR UPDATE` de billetera → débito + `INSERT bets` + fila en `outbox_events` → **COMMIT**.

3 · Confirmación

HTTP 201 con saldo nuevo en decenas de ms (presupuesto total 330–660 ms).

4 · Asíncrono

Outbox Relay publica en Kafka y notifica al apostador sin tocar la ruta crítica.

11 · Decisión 2: Apache Kafka como broker único (ADR-002)

Un solo log particionado cubre streaming de eventos y lotes de liquidación

Múltiples consumidores, un flujo

Cuotas, notificaciones, analítica y auditoría leen el **mismo** log a su propio ritmo con offsets independientes, sin duplicar mensajes ni colas por suscriptor.

Orden y paralelismo por clave

Orden estricto dentro de la partición; clave de negocio (`race_id`, `user_id`, `batch_id`) para distribuir el pico uniformemente.

Dimensionamiento del pico

1 evento canónico por apuesta: ~10.000 ev/s × ~500 B ≈ **5 MB/s** (~150 MB por ráfaga de 30 s): un clúster moderado lo absorbe.

Descarte de RabbitMQ

No por incapacidad técnica, sino porque el modelo de tópicos particionados + consumer groups encaja de forma más compacta y uniforme con todos los flujos del caso, sin tecnologías heterogéneas.

Descarte del dual (RabbitMQ + Kafka)

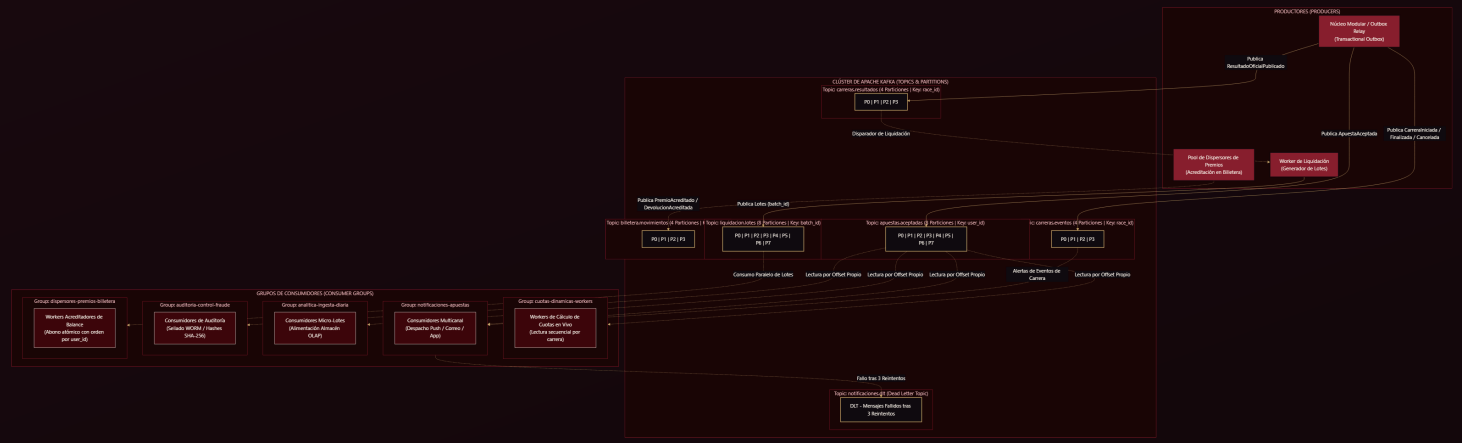
El enunciado exige justificar dos tecnologías; no se demostró ventaja que compense operar, asegurar y monitorizar dos brokers. **Costo de Netflix, problema de cinco medianos.**

Retención operativa ≠ auditoría legal

Kafka retiene ~7 días para absorber ráfagas y replay operativo; la auditoría inalterable de 5 años vive en almacén append-only con WORM y sellado de hashes, alimentado desde Kafka. Los consumidores son idempotentes por `eventId` ante reintentos o rebalanceos.

12 · Topología de eventos y catálogo canónico

Topics, claves de partición y consumer groups del diseño



Topic	Clave	Consumidores
`apuestas.aceptadas`	`user_id`	cuotas, notificaciones, analítica, auditoría
`carreras.eventos`	`race_id`	notificaciones, analítica, consola operador
`carreras.resultados`	`race_id`	worker liquidación, notif., auditoría
`liquidacion.lotes`	`batch_id`	dispersores de premios (pool de workers)
`billetera.movimientos`	`user_id`	notificaciones, auditoría, analítica

Garantía de entrega y DLT

At-least-once (Outbox + reintentos): consumidores idempotentes por `eventId` con `ON CONFLICT DO NOTHING`. Fallos transitorios → topic `retry`; tras 3 intentos → carta muerta `.dlt`.

13 · Estrategia de idempotencia en 3 operaciones críticas

f(f(x)) = f(x): cuatro pulsaciones de «APOSTAR» producen exactamente un débito

1 · Creación de apuesta

- Cliente genera `Idempotency-Key` (UUIDv4) por intención.
- BD: `UNIQUE(user_id, idempotency_key)` en `bets_idempotency`.
- Reintento → colisión atómica → se retorna la respuesta cacheada (HTTP 200) sin cobrar de nuevo.

2 · Pago de premios

- Clave natural por apuesta: `UNIQUE(bet_id, 'PRIZE_PAYOUT')`.
- No se agrega a nivel (carrera, usuario): el usuario puede tener **múltiples apuestas ganadoras legítimas**.
- Reintento de lote → `ON CONFLICT DO NOTHING`.

3 · Replay de eventos Kafka

- Tabla local `processed_events(event_id PK)`.
- `INSERT … ON CONFLICT DO NOTHING` antes de ejecutar lógica.
- Fila ya existente → confirmar offset y descartar en silencio.

Reintento ≠ nueva intención

Misma `Idempotency-Key` = reintento (se responde lo ya confirmado). Nueva apuesta deliberada = **nueva clave**: se procesa como operación independiente con su propio `bet_id`.

Transacción atómica de cierre

El INSERT de la apuesta condiciona `status='ABIERTA'` y `CLOCK_TIMESTAMP() < start_time`: si la espera por bloqueos cruza el cero horario, **0 filas** → **ROLLBACK** → **HTTP 409**. La espera nunca degrada la regla.

14 · Liquidación masiva y algoritmo de mayores restos (ADR-003)

Centavos enteros, sin coma flotante: comisión + premios = bolsa, exactamente al centavo

Por qué este algoritmo (y no otro)

- Meta contable: **comisión + premios = bolsa**, exacto al centavo. Con coma flotante, un millón de apuestas deja fugas acumuladas.
- Hamilton (mayores restos) es **determinista**: mismos datos, mismo reparto, siempre — requisito de auditoría reproducible.
- Pasos: comisión del 10 % con redondeo bancario → bolsa neta por sustracción → parte teórica de cada ganador → base ⌊⌋ → los centavos sobrantes a los **mayores residuos**.
- Desempate por `bet_id` ascendente: dos trabajadores jamás reparten distinto el mismo centavo.
- Cálculo global único en el worker antes de generar lotes: los dispersores **jamás recalculan** restos.

Ejemplo demostrativo

Ganadoras: 33.33 + 33.33 + 33.34 (T_w = 100.00) · B = 100.05

- R = 10.00 → `Bneta = 90.05`
- Teóricos: 30.013665 · 30.013665 · 30.022670 → bases 3.001 · 3.001 · 3.002 (suma 9.004)
- 1 centavo remanente → empate de restos 0.3665 → gana `BET-001`
- Resultado: `30.02 + 30.01 + 30.02`

Comprobación: premios 90.05 + comisión 10.00 = **100.05 = B** ✓ cero fuga de fondos

Trazabilidad: se guarda `settlement_policy_version = 'HAMILTON_LARGEST_REMAINDER_V1'` y los parámetros globales para reproducción forense exacta.

15 · Ausencia de ganadores y devoluciones

Desenlace deportivo válido ≠ incidente operativo: la carrera se liquida, no se cancela

Política del proyecto (P1-DEC-004)

- **Disparo:** resultado oficial publicado con masa ganadora nula (`T_w = 0`).
- **Reembolso íntegro 100 %** de lo apostado a cada participante de esa carrera.
- La casa **renuncia** a su 10 % en este escenario excepcional.
- **Sin rollover:** no se acumula pozo para la siguiente carrera (cierre contable limpio).
- Motivo explícito: `settlement_reason = 'SIN_APUESTAS_GANADORAS'`.

Preservación del ciclo y de la auditoría

- La carrera **NO** pasa por `CANCELADA`: sí se corrió y el ganador cruzó la meta; sus estadísticas permanecen válidas.
- Transita `FINALIZADA → LIQUIDADA` con su motivo propio.
- Dos tipos de devolución distinguibles por el auditor: `REFUND_RACE_CANCELLED` (incidente) vs. `REFUND_NO_WINNERS` (desenlace).
- Idempotencia: `UNIQUE(bet_id, 'REFUND_NO_WINNERS')`.
- Bolsa vacía (`B = 0`): `LIQUIDADA` con motivo `BOLSA_VACIA`, sin divisiones por cero.

Cancelación (contraste)

La cancelación es transición de excepción **desde cualquier estado previo a FINALIZADA** con devolución íntegra del 100 % — un incidente operativo (clima, pista), no un resultado deportivo.

16 · Manejo de cargas masivas (10.000 apuestas/s)

Escalamiento asimétrico: lo que crece en el pico y lo que se aparta del mismo

Escalan agresivamente en el pico

- **API Gateway:** 100.000 conexiones TLS, rate limiting y anti-bot.
- **`core-api`:** réplicas stateless (CPU/memoria).
- **Topic `apuestas.aceptadas`:** particiones dimensionadas.
- **OLTP:** IOPS elásticos NVMe + réplicas de lectura.

No escalan en el pico (y está bien)

- **Liquidación:** la carrera aún no terminó; permanece inactiva.
- **Analítica OLAP:** congela la ingesta pesada; tolera minutos.
- **Notificaciones:** drenan a su ritmo, desacopladas por colas.
- **Consolas operativas:** < 1 operación por segundo.

Presupuesto de latencia

Cliente↔gateway 150–300 ms + gateway 5–10 ms + red interna 2–5 ms + transacción BD 15–40 ms + respuesta ≈ 5 ms → **total 330–660 ms** (< 1.000 ms). Protegido por cero saltos RPC síncronos y outbox.

Liquidación < 15 min

1.000.000 apuestas en 900 s exige ≥ 1.111 acreditaciones/s: cálculo global < 10 s, ~1.000–2.000 lotes, 10 workers concurrentes con batch inserts → **6–9 minutos estimados**.

Durabilidad

WAL con commit síncrono + réplica síncrona: **RPO = 0 s** (cero pérdida confirmada), **RTO < 30 s** con failover automático; PITR continuo hacia objetos.

17 · Análisis de fallos: casos A, B y C

La respuesta depende de qué invariante gobierna el subsistema caído

Caso A · Notificaciones caídas → Sí, se sigue apostando

Son capacidad de **consistencia eventual**, desacopladas por Kafka. Los eventos `ApuestaAceptada` se acumulan seguros en el topic; el usuario ya recibió su HTTP 201 en pantalla. Al recuperarse, el servicio reanuda desde su último offset y despacha lo pendiente.

Caso B · Billetera caída → NO, se rechaza/bloquea

Verificar saldo y debituar es **precondición indivisible** de toda apuesta (consistencia fuerte). Seguir operando equivaldría a apostar a crédito: saldos negativos incontrolados y modelo Pari-Mutuel violado.

Caso C (opcional) · Resultado mal interpretado tras la foto de llegada

1. Congelar: el operador registra la rectificación; se frena cualquier lote en curso (`RESULTADO_EN_RECTIFICACION`).

2. No mutar: jamás se borra el error: se recalcula con `settlement_version = 2` y se sella el evento `ResultadoRectificado`.

3. Compensar: premios legítimos al ganador real (clave compuesta con versión); al beneficiado erróneo se le descuenta hasta 0 (`LEAST(balance, x)`) — jamás saldo negativo — y el insoluto se registra como pérdida operativa asumida.

18 · Seguridad, RBAC y segmentación de red

Cuatro roles, mínimo privilegio y perímetro público mínimo

Operación	Apostador	Operador	Admin	Auditor
Emitir apuesta / consultar saldo propio	✓	✗	✗	✗
Programar carreras / publicar resultado	✗	✓	✗	✗
Bloquear usuarios / gestionar roles	✗	✗	✓	✗
Consultar auditoría	✗	✗	solo métricas	✓ **solo lectura**
Modificar saldos o apuestas directamente	✗	✗	✗	✗

Perímetro y anti-abuso

Rate limiting multinivel: 5 apuestas/s por usuario (ráfaga 10), 50 consultas/min por IP; HTTP 429 + `Retry-After`; detección de bots y bloqueo de cuentas múltiples en los últimos 10 s al cierre.

Identidad y cifrado

JWT stateless (EdDSA/RS256, 15–30 min) + refresh rotativo; Argon2id/bcrypt; TLS 1.3 externo, mTLS interno, AES-256 en reposo y secretos en KMS (cero secretos en el repositorio).

DMZ vs. VPC

Solo CDN y Gateway son públicos. Cómputo (core, workers), Kafka, OLTP, OLAP y auditoría WORM residen en subredes privadas sin IP accesible desde Internet.

19 · Gobernanza de costos y evolución ante crecimiento 100×

La pregunta final: qué implementaríamos hoy y cómo evoluciona sin reescrituras

Hoy (Fase 100k usuarios / 10k apuestas/s)

- 1 núcleo modular replicable + 1 Kafka administrado + 3 workers ligeros.
- BD administrada con réplica; cómputo elástico que se reduce al mínimo en meses intersemestrales.
- Sin Sagas, sin service mesh, sin segundo broker: complejidad solo donde el requisito la exige.

Respuesta a la pregunta del enunciado

«Implementaríamos una **Arquitectura Híbrida** hoy: núcleo transaccional con consistencia ACID local y Kafka para el desacople real. Crecer 100× **no** implica reescribir: cada evolución responde a una señal empírica de cuello de botella.»

Evolución	Señal disparadora	Mecanismo
1 · Sharding de BD	WAL saturado; IOPS primario no supera ~15.000 escrituras/s	Shards por `user_id`: la apuesta toca un solo shard → se preserva la atomicidad local sin transacciones distribuidas.
2 · Segregar billetera	Equipos independientes o regulación contable externa	**Única evolución que destruye ACID local** (exige Saga); no es obligatoria ni recomendable al inicio.
3 · Tiered storage	El volumen de eventos satura los NVMe del clúster	Descarga de segmentos fríos a objetos sin tocar la memoria del broker.
4 · Edge (PoPs)	Latencia geográfica > 400 ms para clientes lejanos	Terminación TLS y rate limiting cercanos al usuario.

20 · CONCLUSIONES

Tres lecciones de diseño y cierre de la sustentación

1 · La complejidad debe justificarse

«Una buena arquitectura no es la que usa más tecnologías, sino la que usa la complejidad necesaria.» 20 microservicios y dos brokers para carreras de cinco medianos habrían sido costosos, frágiles e innecesarios.

2 · La coherencia contable es innegociable

Transacción local atómica + bloqueo por fila + claves idempotentes + mayores restos = integridad matemática al centavo: sin saldos negativos ni premios dobles, aunque alguien pulse cuatro veces.

3 · El desacoplamiento da resiliencia

Notificaciones y analítica fuera de la ruta crítica: la caída de un proveedor externo o una consulta pesada jamás tocan la admisión de apuestas.

Gracias

Salchipapa Maker Racing S.A.S. · La Gran Liga de Apuestas de Medianos EIA · Octubre 2026